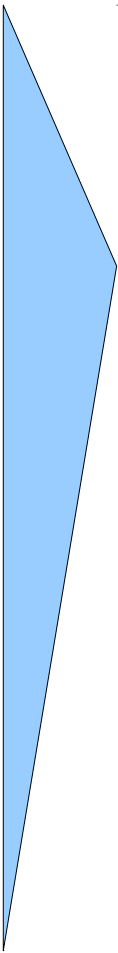
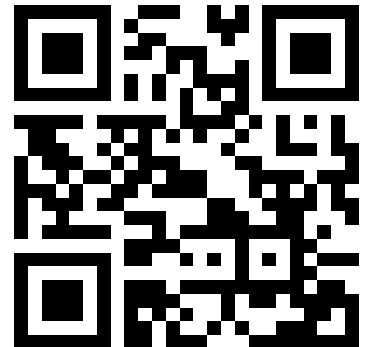


Lecture Overview

- 
- Introduction, Microcontroller basics
 - 32-bit Microcontroller technology
 - ARM Cortex M Architecture, (CM0 CM3 CM4 CM7)
 - Cortex M Microcontroller Instruction Set
 - Exception Handling, Interrupts
 - Real-time Operating Systems (RTOS)
 - Memory Management, protected memory, MPU
 - Implementing Calculations, Data Formats, FPU
 - I/O, Device drivers
 - Application Design
 - Case Studies, Advanced Design Patterns

<https://skript.eit.h-da.de/ams/>



Cortex M0 Instruction set

- Smallest and most simple instruction set within the Cortex MPU Family
- Subset of the ARMV6 Thumb instructions
- Almost only 16bit instructions



VABS	VADD	VCMP	VCMPPE	VCVT	VCVTR	VDIV	VLDM
VLDR	VMLA	VMLS	VMOV	VMRS	VMSR	VMUL	VNEG
VNMLA	VMMLS	VNMUL	VPOP	VPUSH	VSQRT	VSTM	VSTR
VSUB	VFMA	VFMS	VFNMA	VFNMS			

Cortex-M4 FPU

PKH	QADD	QADD16	QADD8	QASX	QDADD	QDSUB	QSAX
QSUB	QSUB16	QSUB8	SADD16	SADD8	SASX	SEL	SHADD16
SHADD8	SHASX	SHSAX	SHSUB16	SHSUB8	SMLABB	SMLABT	SMLATB
SMLATT	SMLAD	SMLALBB	SMLALBT	SMLALTB	SMLALTT	SMLALD	SMLAWB
SMLAWT	SMLS	SMLS	SMMLA	SMMLS	SMMUL	SMUAD	SMULBB

ADC	ADD	ADR	AND	ASR	B
CLZ	BFC	BF	BIC	CDP	CLREX
CBNZ	CBZ	CMN	CMP	DBG	EOR
LDMIA	LDMDB	LDR	LDRB	LDRBT	LDRD
LDREX	LDREXB	LDREXH	LDRH	LDRHT	LDRSB
LDRSBT	LDRSHT	LDRSH	LDRT	MCR	LSL
LSR	MCRR	MLS	MLA	MOV	MOVT
MRC	MRRC	MUL	MVN	NOP	ORN
ORR	PLD	PLDW	PLI	POP	PUSH
RBIT	REV	REV16	REVSH	ROR	RRX

BKPT	BLX	ADC	ADD	ADR
BX	CPS	AND	ASR	B
DMB		BL		BIC
DSB	CMN	CMP		EOR
ISB	LDR	LDRB	LDM	
MRS	LDRH	LDRSB	LDRSH	
MSR	LSL	LSR	MOV	
NOP	REV	MUL	MVN	ORR
REV16	REVSH	POP	PUSH	ROR
SEV	SXTB	RSB	SBC	STM
SXTH	UXTB	STR	STRB	STRH
UXTH	WFE	SUB	SVC	TST
WFI	YIELD			

Cortex-M0/M0+/M1

RSB	SBC	SBFX
SDIV	SEV	SMLAL
SMULL	SSAT	STC
STMIA	STMDB	STR
STRB	STRBT	STRD
STREX	STREXB	STREXH
STRH	STRHT	STRT
SUB	SXTB	SXTH
TBB	TBH	TEQ
TST	UBFX	UDIV
UMLAL	UMULL	USAT
UXTB	UXTH	WFE
WFI	YIELD	IT

Cortex-M3

SMULBT	SMULTT
SMULTB	SMULWT
SMULWB	SMUSD
SSAT16	SSAX
SSUB16	SSUB8
SXTAB	SXTAB16
SXTAH	SXTB16
UADD16	UADD8
UASX	UHADD16
UHADD8	UHASX
UHSAX	UHSUB16
UHSUB8	UMAAL
UQADD16	UQADD8
UQASX	UQSAX
UQSUB16	UQSUB8
USAD8	USADA8
USAT16	USAX
USUB16	USUB8
UXTAB	UXTAB16
UXTAH	UXTB16

Cortex-M4

How to develop an Instruction Set

Identify all needed Instructions

DO

Check frequency of instruction occurrence in a typical application

Assign OP-Codes

Run tests with typical application code

WHILE (not satisfied with resulting performance and code density)

CISC RISC VLIW

Complex Instruction Set Computer

Example: Intel x86

- Many powerful instructions

- Small register count

- Irregular / unsymmetrical instruction set

Reduced Instruction Set Computer

Examples: AVR, ARM

- Few instructions

- Many symmetric general registers

- Well suited for compiled code

Very Large Instruction Word

- All addresses can be coded into a single instruction

- Parallel computing built into the ISA

- Presently only found in computing cluster processors

1 Op 2 Op 3 Op Machines

Example Problem:

Memory(loc3) = Memory(loc1) + Memory(loc2)

1 Operand Machine:
(Accumulator Machine)

Load (loc1)
Add (loc2)
Store (loc3)

2 Operand Machine:

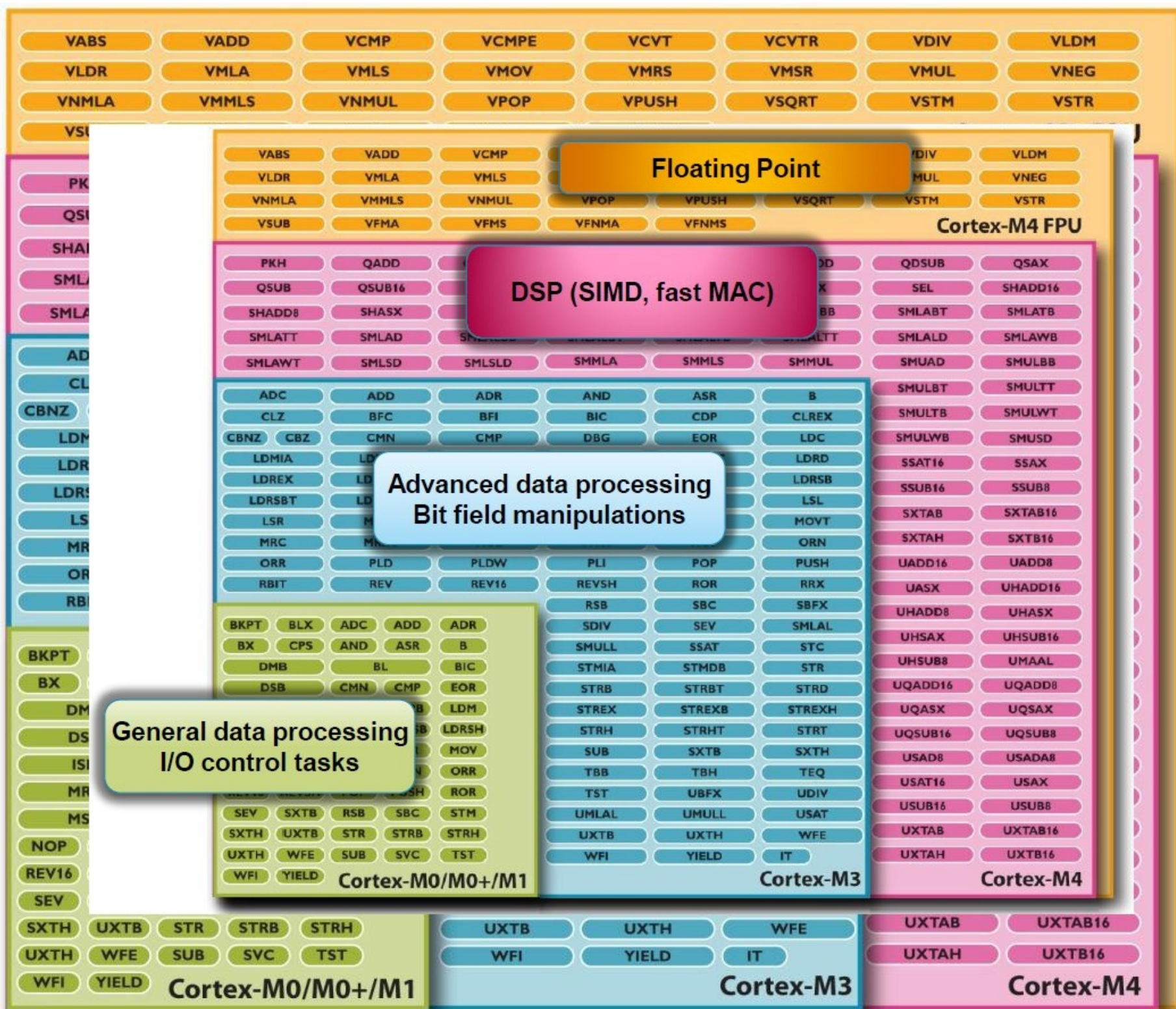
Load R1, (loc1)
Load R2, (loc2)
Add R1 += R2
Store (loc3), R1

3 Operand Machine:

Load R1, (loc1)
Load R2, (loc2)
Add R3 = R1 + R2
Store (loc3), R3

Instruction Groups

- Load and Store
- Arithmetical and logical
- Jump and branch
- Stacking / unstacking (PUSH / POP)
- Processor control
- Vector SIMD instructions
- Floating-Point instructions



CM0 Instructions

A closer look ...



Register Set

Name		Functions (and banked registers)	
R0		General-purpose register	Low registers
R1		General-purpose register	
R2		General-purpose register	
R3		General-purpose register	
R4		General-purpose register	
R5		General-purpose register	
R6		General-purpose register	
R7		General-purpose register	High registers
R8		General-purpose register	
R9		General-purpose register	
R10		General-purpose register	
R11		General-purpose register	
R12		General-purpose register	
R13 (MSP)	R13 (PSP)	Main Stack Pointer (MSP), Process Stack Pointer (PSP)	
R14		Link Register (LR)	
R15		Program Counter (PC)	

Data types in memory

Byte	8 bits
Halfword	16 bits
Word	32 bits
Doubleword	64 bits
Float	32 bits
Double	64 bits

Arithmetic Flags

Processor Status Register High Bits

N, bit [31]

Negative condition code flag.

Set to bit [31] of the result of the instruction.

If the result is regarded as a two's complement signed integer, then N is set to 1 if the result is negative and set to 0 if it is positive or zero.

Z, bit [30]

Zero condition code flag.

Set to 1 if the result of the instruction is zero, and to 0 otherwise.

A result of zero often indicates an equal result from a comparison.

C, bit [29]

Carry condition code flag.

Set to 1 if the instruction results in a carry condition,

For example an unsigned overflow on an addition.

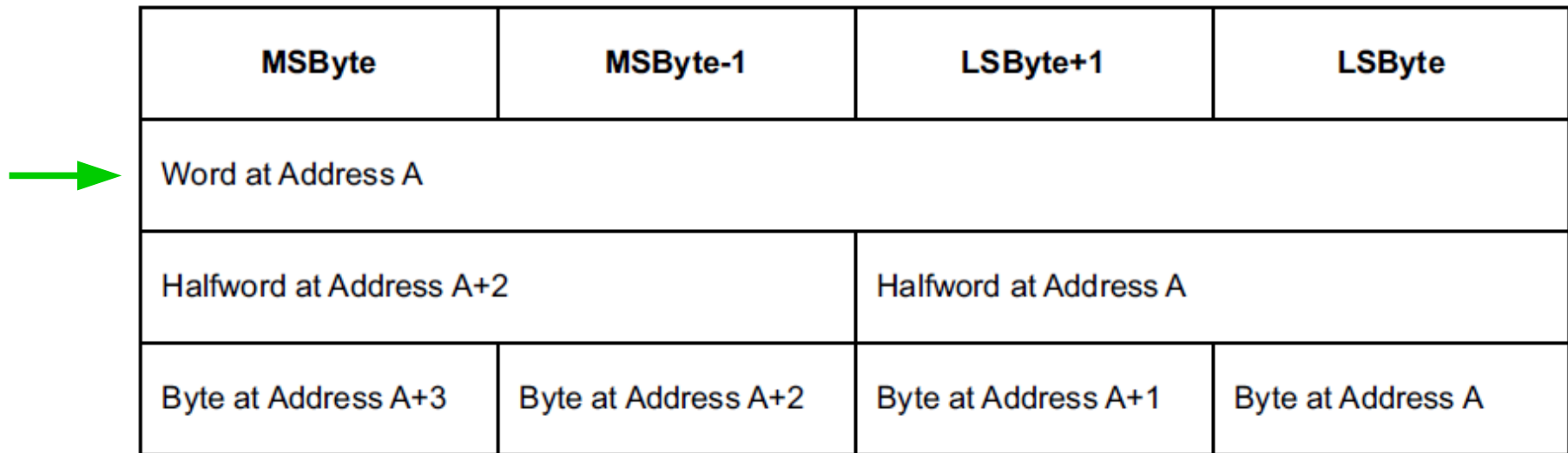
V, bit [28]

Overflow condition code flag.

Set to 1 if the instruction results in an overflow condition,

for example a signed overflow on an addition.

Little Endian Memory Model



→ An **address** stands for the location of a **byte** in memory

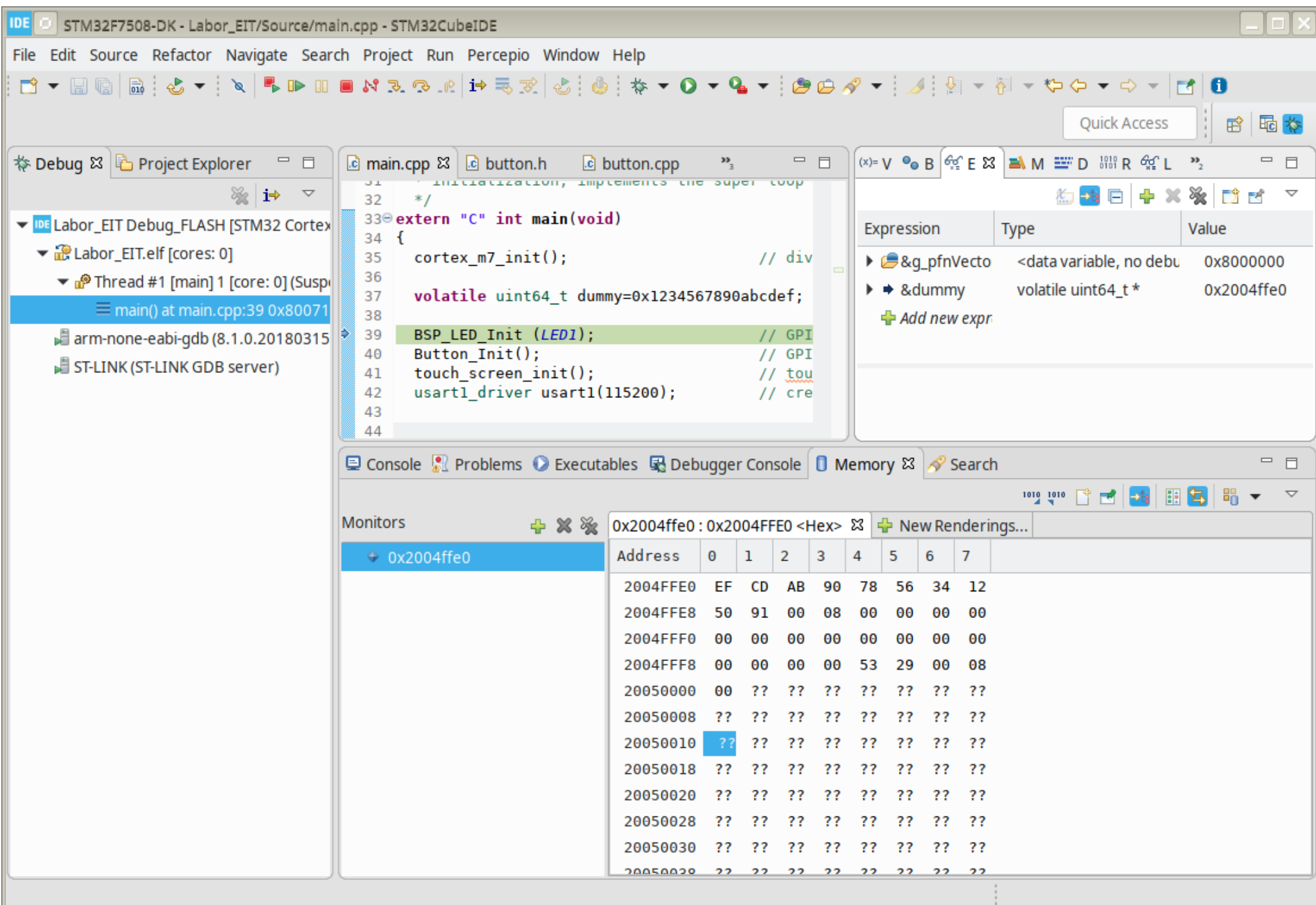
Aligned halfwords have even addresses (equiv. to LSB=0)

Aligned **words** have addresses with the lowest two bits = zero

```
uint64_t test = 0x123456789abcdef
```

```
At 0x00000000:  ef cd ab 89 67 45 23 12 ...
```

Example: 64 bit HEX Number



Opcode Size: 16 bits preferred !

$$\begin{array}{ccccccc} \text{Add} & \text{memory[a]} & = & \text{memory[b]} & + & \text{memory[c]} & \\ \uparrow & \uparrow & & \uparrow & & \uparrow & \\ 7 & + & 32 & + & 32 & + & 32 & = 103 \text{ bit} \end{array}$$

$$\begin{array}{ccccccc} \text{Add} & \text{R1} & & = & \text{R2} & + & \text{R3} & \\ \uparrow & \uparrow & & & \uparrow & & \uparrow & \\ 7 & + & 3 & + & 3 & + & 3 & = 16 \text{ bit} \end{array}$$

Load and store architecture, $\rightarrow$ “REGISTERS” !
Only few bits are necessary to address registers.

The number of addressable registers is an important design decision !

Move

8-bit immediate	MOVS Rd, #<imm>	1
Lo to Lo	MOVS Rd, Rm	1
Any to Any	MOV Rd, Rm	1
Any to PC	MOV PC, Rm	2

Description

Assembler

Cycles

Data processing Instructions

- Consist of :

- Arithmetic: **ADD** **ADC** **SUB** **SBC** **RSB** **RSC**
- Logical: **AND** **ORR** **EOR** **BIC**
- Comparisons: **CMP** **CMN** **TST** **TEQ**
- Data movement: **MOV** **MVN**

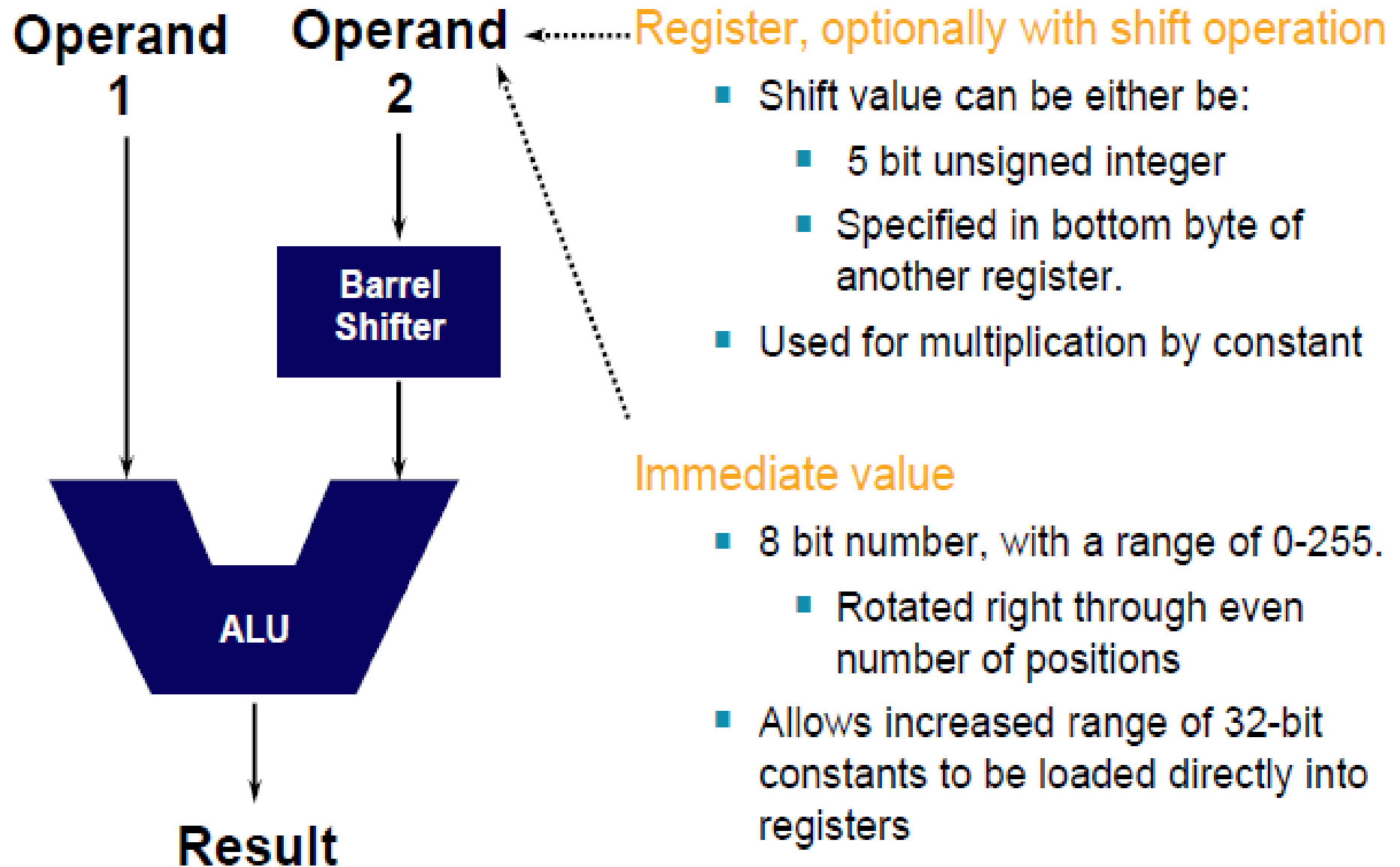
- These instructions only work on registers, NOT memory.

- Syntax:

<Operation>{<cond>}{S} Rd, Rn, Operand2

- Comparisons set flags only - they do not specify Rd
- Data movement does not specify Rn
- Second operand is sent to the ALU via barrel shifter.

Using a Barrel Shifter: The 2nd Operand



Add

3-bit immediate	ADDS Rd, Rn, #<imm>	1
All registers Lo	ADDS Rd, Rn, Rm	1
Any to Any	ADD Rd, Rd, Rm	1
Any to PC	ADD PC, PC, Rm	2
8-bit immediate	ADDS Rd, Rd, #<imm>	1
With carry	ADCS Rd, Rd, Rm	1
Immediate to SP	ADD SP, SP, #<imm>	1
Form address from SP	ADD Rd, SP, #<imm>	1
Form address from PC	ADR Rd, <label>	1

Description

Assembler

Cycles

Examples of ADD

ADD comes in different versions:

ADDS <Rd>,<Rn>,#<imm3>

Add to Rd Rn plus 3bit immediate

S switches setting of flags on

ADDS <Rdn>,#<imm8>

Add 8bit immediate to Rdn

S switches setting of flags on

ADDS <Rd>,<Rn>,<Rm>

Add Rm to Rm and store result in Rd

S switches setting of flags on

ADD <Rd>,SP,#<imm8>

Add imm8 to SP and store result in Rd

ADD <Rd>, SP, <Rm>

Add SP to Rm and store result in Rd

Subtract

Lo and Lo	SUBS Rd, Rn, Rm	1
3-bit immediate	SUBS Rd, Rn, #<imm>	1
8-bit immediate	SUBS Rd, Rd, #<imm>	1
With carry	SBCS Rd, Rd, Rm	1
Immediate from SP	SUB SP, SP, #<imm>	1
Negate	RSBS Rd, Rn, #0	1

Description

Assembler

Cycles

Compare

Compare	CMP Rn, Rm	1
Negative	CMN Rn, Rm	1
Immediate	CMP Rn, #<imm>	1

Description

Assembler

Cycles

Multiply

Multiply

MULS Rd, Rm, Rd

1 or 32

Description

Assembler

Cycles

Logical

AND	ANDS Rd, Rd, Rm	1
Exclusive OR	EORS Rd, Rd, Rm	1
OR	ORRS Rd, Rd, Rm	1
Bit clear	BICS Rd, Rd, Rm	1
Move NOT	MVNS Rd, Rm	1
AND test	TST Rn, Rm	1

Description

Assembler

Cycles

Shift Rotate

Logical shift left by immediate	LSLS Rd, Rm, #<shift>	1
Logical shift left by register	LSLS Rd, Rd, Rs	1
Logical shift right by immediate	LSRS Rd, Rm, #<shift>	1
Logical shift right by register	LSRS Rd, Rd, Rs	1
Arithmetic shift right	ASRS Rd, Rm, #<shift>	1
Arithmetic shift right by register	ASRS Rd, Rd, Rs	1
Rotate right by register	RORS Rd, Rd, Rs	1

Description

Assembler

Cycles

Load

Word, immediate offset	LDR Rd, [Rn, #<imm>]	2 or 1
Halfword, immediate offset	LDRH Rd, [Rn, #<imm>]	2 or 1
Byte, immediate offset	LDRB Rd, [Rn, #<imm>]	2 or 1
Word, register offset	LDR Rd, [Rn, Rm]	2 or 1
Halfword, register offset	LDRH Rd, [Rn, Rm]	2 or 1
Signed halfword, register offset	LDRSH Rd, [Rn, Rm]	2 or 1
Byte, register offset	LDRB Rd, [Rn, Rm]	2 or 1
Signed byte, register offset	LDRSB Rd, [Rn, Rm]	2 or 1
PC-relative	LDR Rd, <label>	2 or 1
SP-relative	LDR Rd, [SP, #<imm>]	2 or 1
Multiple, excluding base	LDM Rn!, {<loreglist>}	1+N
Multiple, including base	LDM Rn, {<loreglist>}	1+N

Store

Word, immediate offset	STR Rd, [Rn, #<imm>]	2 or 1
Halfword, immediate offset	STRH Rd, [Rn, #<imm>]	2 or 1
Byte, immediate offset	STRB Rd, [Rn, #<imm>]	2 or 1
Word, register offset	STR Rd, [Rn, Rm]	2 or 1
Halfword, register offset	STRH Rd, [Rn, Rm]	2 or 1
Byte, register offset	STRB Rd, [Rn, Rm]	2 or 1
SP-relative	STR Rd, [SP, #<imm>]	2 or 1
Multiple	STM Rn!, {<loreglist>}	1+N

Push / Pop

Push	<code>PUSH {<loreglist>}</code>	1+N
Push with link register	<code>PUSH {<loreglist>, LR}</code>	1+N
Pop	<code>POP {<loreglist>}</code>	1+N
Pop and return	<code>POP {<loreglist>, PC}</code>	3+N

Description

Assembler

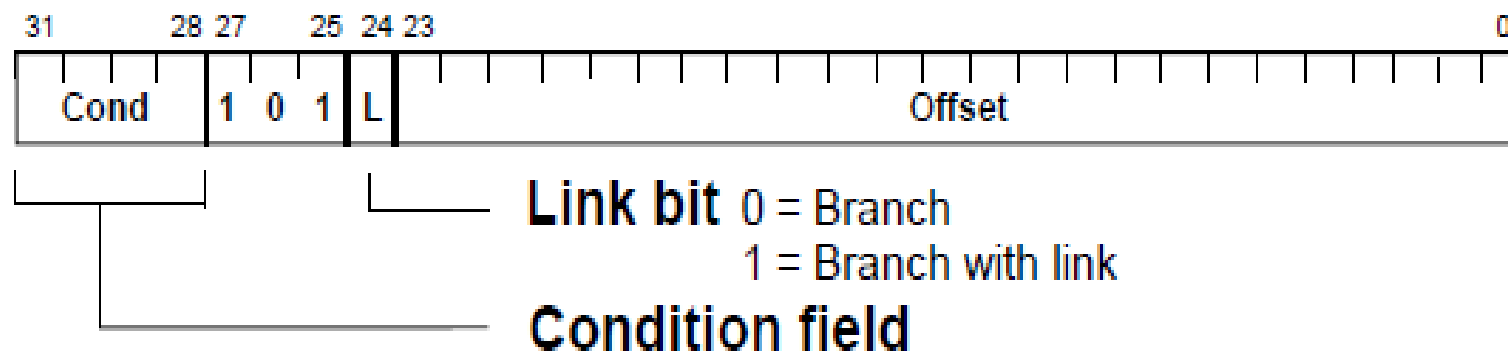
Cycles

Branch

Conditional	B<cc> <label>	1 or 2
Unconditional	B <label>	2
With link	BL <label>	3
Description	Assembler	Cycles

Branch instructions

- Branch : `B{<cond>} label`
- Branch with Link : `BL{<cond>} subroutine_label`



- The processor core shifts the offset field left by 2 positions, sign-extends it and adds it to the PC
 - ± 32 Mbyte range
 - How to perform longer branches?

Manipulate Data

Signed halfword to word	SXTH Rd, Rm	1
Signed byte to word	SXTB Rd, Rm	1
Unsigned halfword	UXTH Rd, Rm	1
Unsigned byte	UXTB Rd, Rm	1
Bytes in word	REV Rd, Rm	1
Bytes in both halfwords	REV16 Rd, Rm	1
Signed bottom half word	REVSH Rd, Rm	1

Description

Assembler

Cycles

Interrupt / Processor State

Supervisor Call	SVC #<imm>	
Disable interrupts	CPSID i	1
Enable interrupts	CPSIE i	1
Read special register	MRS Rd, <specreg>	3
Write special register	MSR <specreg>, Rn	3
Breakpoint	BKPT #<imm>	

Description

Assembler

Cycles

Multi – Processor support

Instruction synchronization	ISB	3
Data memory	DMB	3
Data synchronization	DSB	3
Description	Assembler	Cycles

32 bit commands

Special

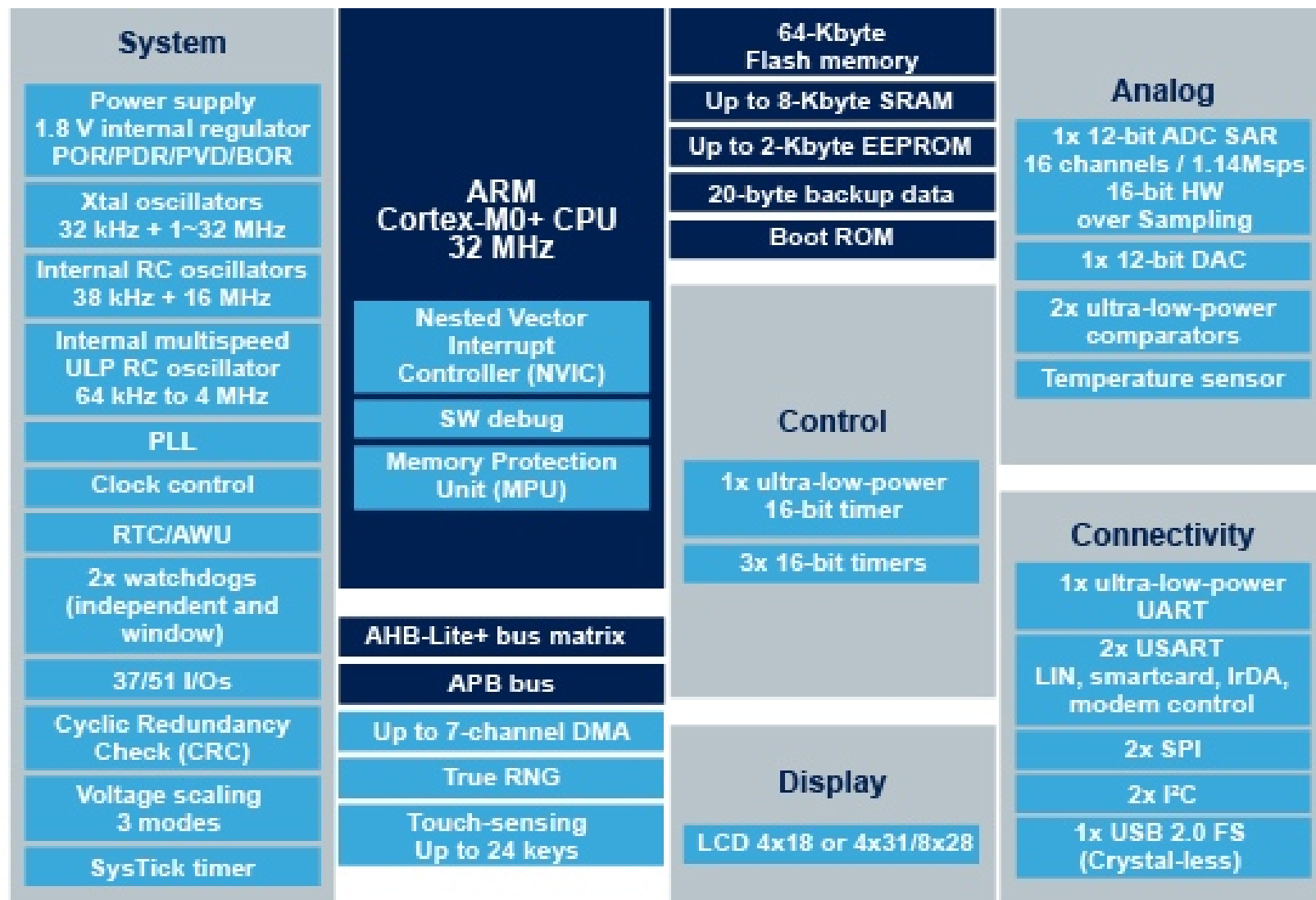
Send event	SEV	1	
Wait for event	WFE	2	Almost equal OP
Wait for interrupt	WFI	2	
Yield	YIELD	1	Not implemented
No operation	NOP	1	
Description	Assembler	Cycles	

IOT Example

Smart Energy Meter



Example: STM32L053



Instruction Coding



0x12345678 32 bit

0xabcd1234 2 * 16bit

Opcode (~instruction name)

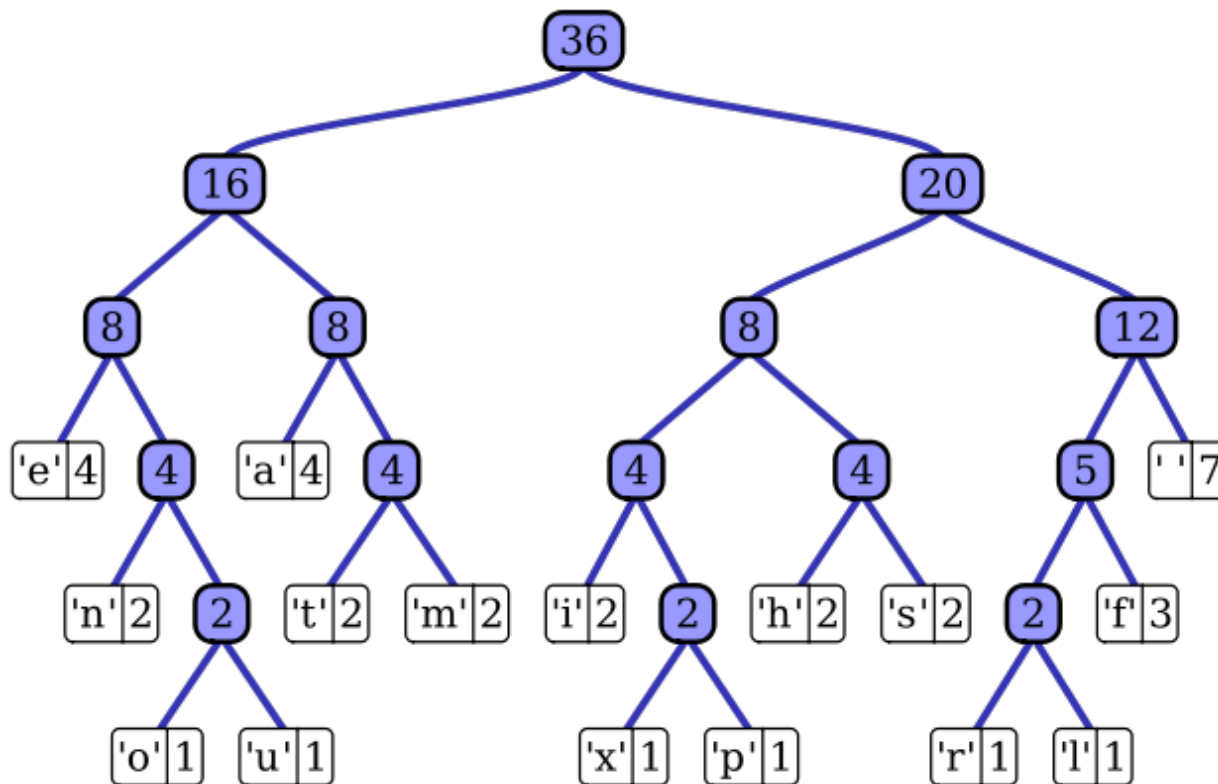
Target register(s)

Memory address,
Address offset

Immediate data

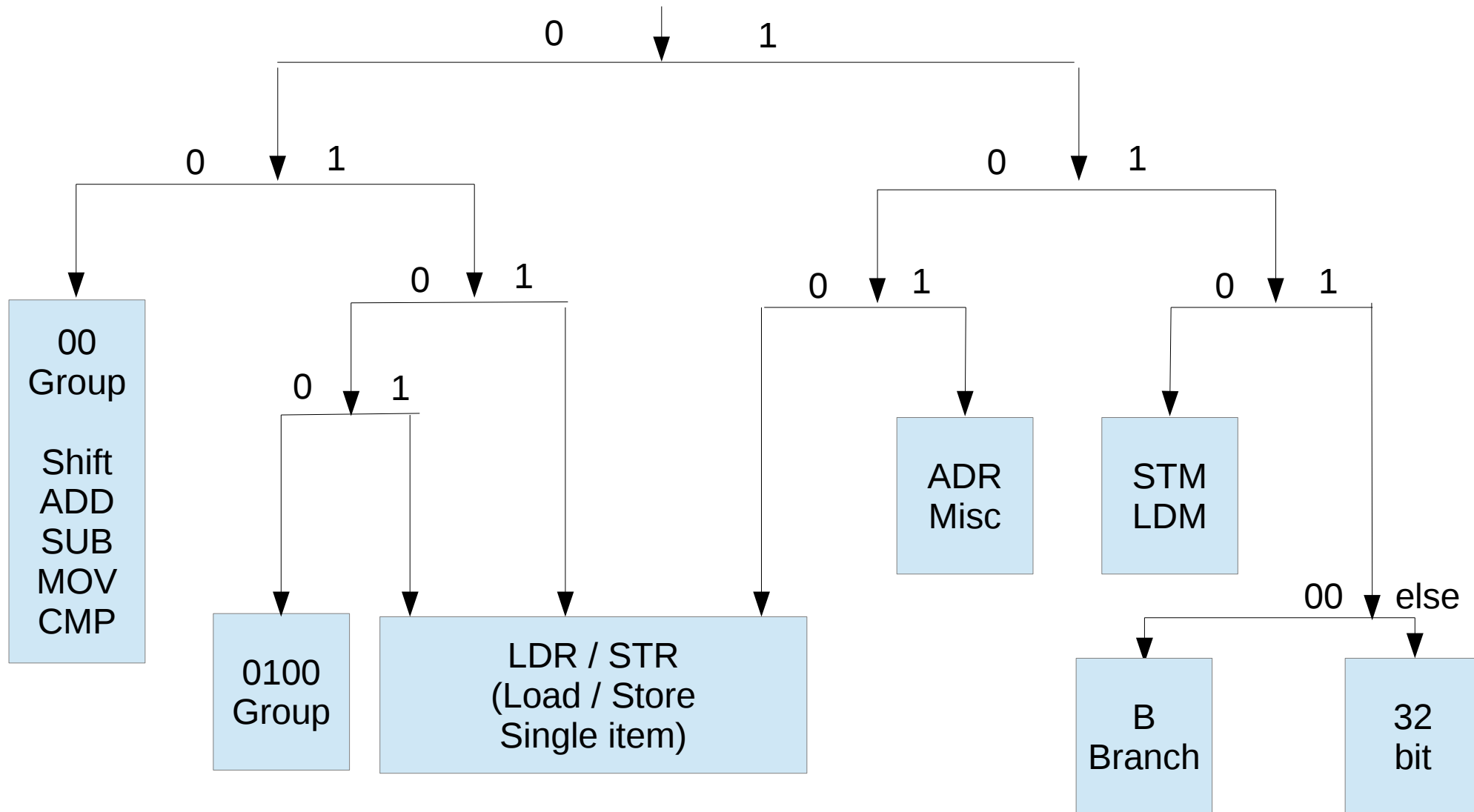
Huffman Encoding

Example text: *this is an example of a huffman tree*



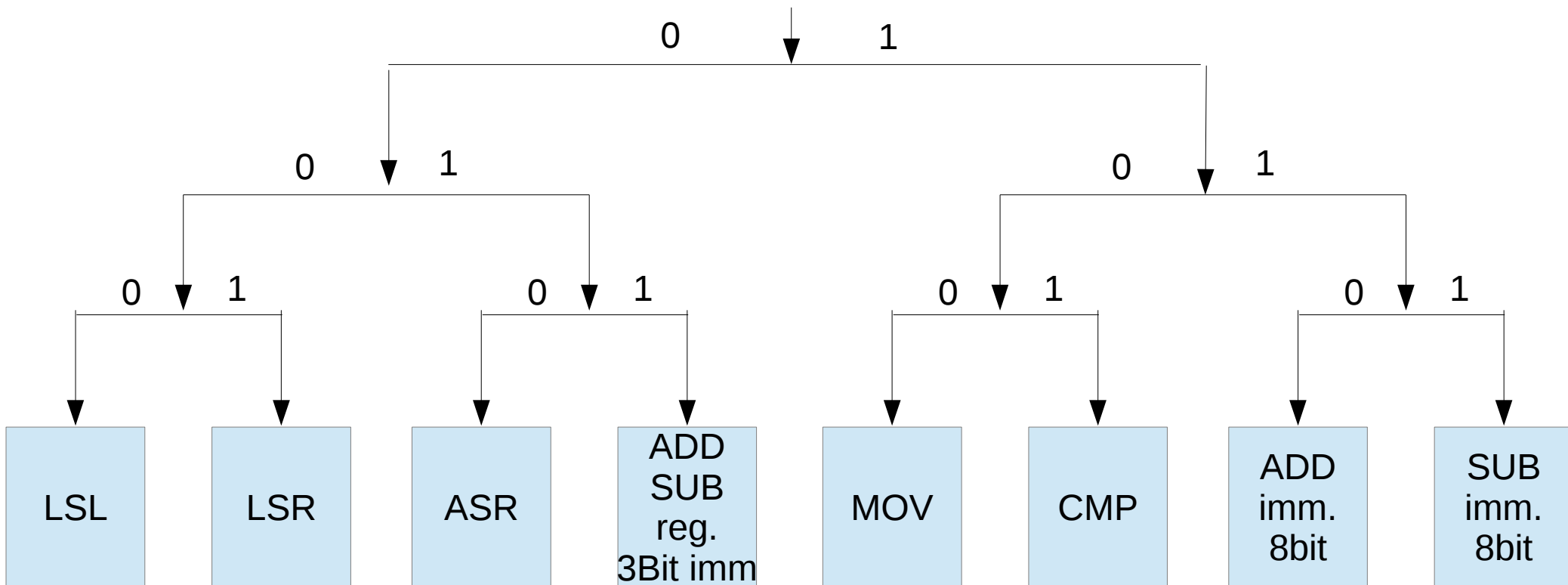
Thumb Instruction Tree

Root



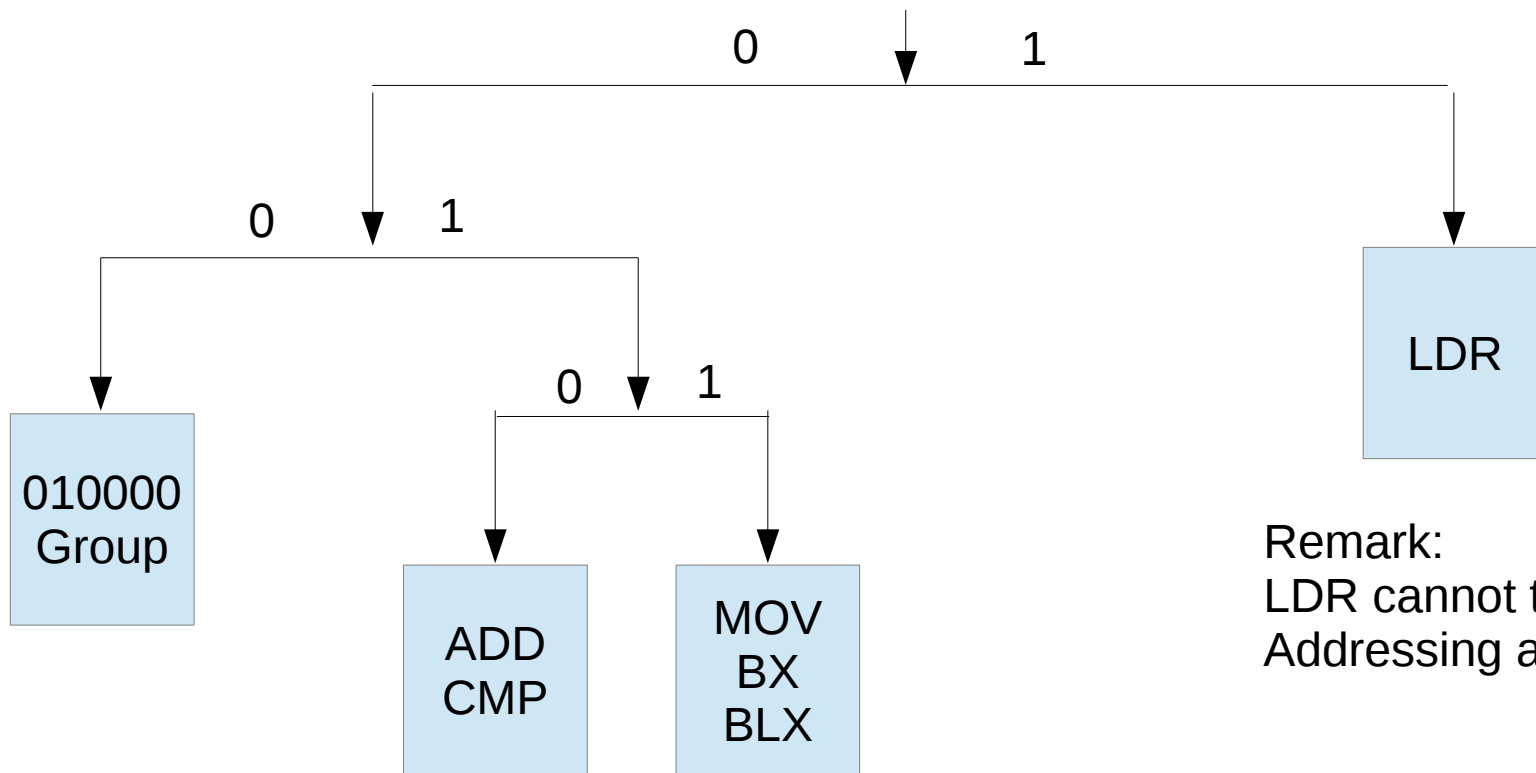
Thumb Instruction Tree

00-Branch



Thumb Instruction Tree

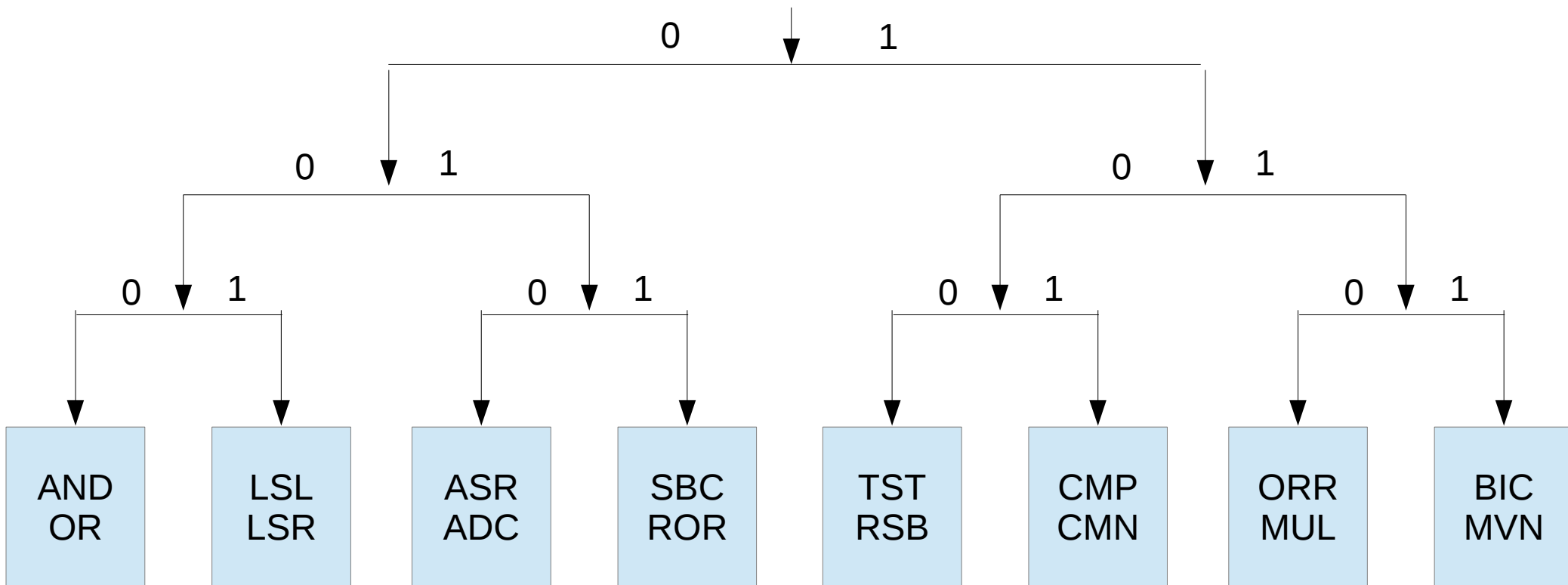
0100-Group



Remark:
LDR cannot take an address !
Addressing always rel. to PC.

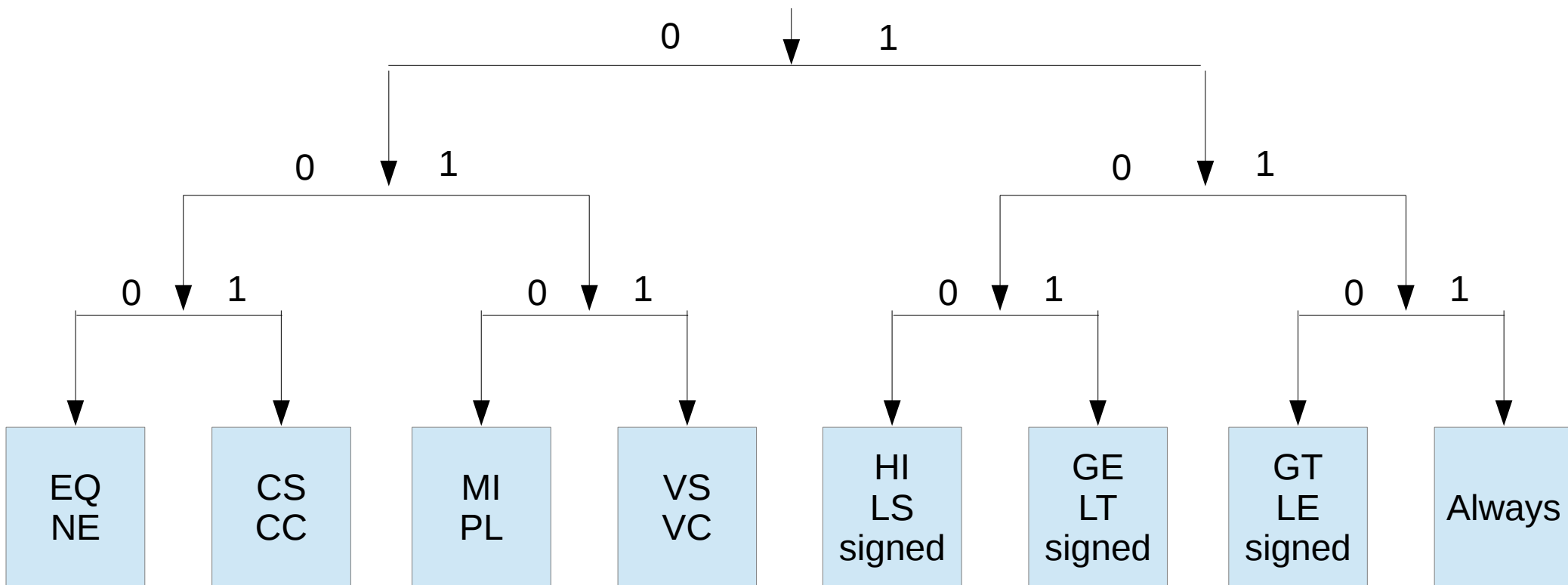
Thumb Instruction Tree

010000-Group



ARM Condition Codes

4-bit encoding



Flags = { Z = zero, C = carry, N = negative, V = oVerflow }

32bit Thumb Instructions

1111 Block

MSR	Move to special register
MRS	Move from special register
BL	Branch with link
DSB	Data synchronization barrier
DMB	Data memory barrier
ISB	Instruction synchronization barrier

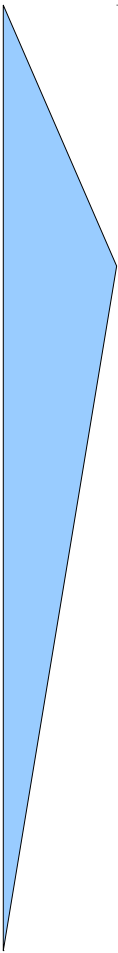
Remarks

- Thumb Instructions are pretty RISCy
Only essential instructions are implemented.
Some instructions have many tricky modes.
- There are no LOAD / STORE extended instructions !
Those would need too much ROM space.
- Addressing is always done via a register,
optionally modified by an offset (3bit, 8bit, index-register).
- Instruction set extensions are possible but “expensive”
as they will result in long OP-Codes.

References

- ARMV6M Architecture Reference manual
- ARMV7M Architecture Reference manual
- Thumb2 Quick Reference

Lecture Overview

- 
- Introduction, Microcontroller basics
 - 32-bit Microcontroller technology
 - ARM Cortex M Architecture, (CM0 CM3 CM4 CM7)
 - Cortex M Microcontroller Instruction Set
 - Exception Handling, Interrupts
 - Real-time Operating Systems (RTOS)
 - Memory Management, protected memory, MPU
 - Implementing Calculations, Data Formats, FPU
 - I/O, Device drivers
 - Application Design
 - Case Studies, Advanced Design Patterns

<https://skript.eit.h-da.de/ams/>

